- *Razor view*—Razor views (also called Razor templates) are used to generate HTML. They are typically used in the final stage of a Razor Page to generate the HTML response to send back to the user.

In the previous four chapters, I covered a whole cross section of Razor Pages, including the Model-View-Controller (MVC) design pattern, the Razor Page `PageModel`, page handlers, routing, and binding models. This chapter covers the last part of the MVC pattern: using a view to generate the HTML that's delivered to the user's browser.

In ASP.NET Core, views are normally created using the *Razor* markup syntax (sometimes described as a templating language), which uses a mixture of HTML and C# to generate the final HTML. This chapter covers some of the features of Razor and how to use it to build the view templates for your application. Generally speaking, users will have two sorts of interactions with your app: they'll read data that your app displays, and they'll send data or commands back to it. The Razor language contains several constructs that make it simple to build both types of applications.

When displaying data, you can use the Razor language to easily combine static HTML with values from your `PageModel`. Razor can use C# as a control mechanism, so adding conditional elements and loops is simple—something you couldn't achieve with HTML alone.

The normal approach to sending data to web applications is with HTML forms. Virtually every dynamic app you build will use forms; some applications will be pretty much nothing *but* forms! ASP.NET Core and the Razor templating language include *Tag Helpers* that make generating HTML forms easy.

NOTE You'll get a brief glimpse of Tag Helpers in section 17.1, but I explore them in detail in chapter 18.